Similarity-Based Duplicate Record Detection Using String Matching and Machine Learning

Bonagani Prathusha [1*]
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad, Telangana, India
Email: prathushabonagani@klh.edu.in

Allipuram Akhil [2]
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad, Telangana, India
Email: 2420030540@klh.edu.in

D. Surya [3]
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad, Telangana, India
Email: 2420030440@klh.edu.in

P. Ruthvik Reddy [4]
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Bowrampet, Hyderabad, Telangana, India
Email: 2420030546@klh.edu.in

***Abstract*—Duplicate customer records cause trouble that is easy to overlook. A customer's history ends up spread over several rows, counts come out too high, and any analysis built on the data becomes less reliable. In this paper we build and test a similarity-based system for detecting duplicate records. It covers the full workflow: text normalisation, blocking, multi-field similarity scoring, supervised classification, clustering of duplicates and an interactive dashboard. Every candidate pair is described by Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine and exact-match features, and we compare Logistic Regression, Random Forest and XGBoost on these same features. Our test data is a generated set of 503 records (350 originals and 153 injected duplicates). Blocking brought the 126,253 possible pairs down to 605 candidates, 172 of which were flagged as duplicates, which left 357 representative records, a 29.0% reduction. All three classifiers reported an accuracy of 0.995, precision of 1.000, recall of 0.981 and F1 of 0.990 on 182 evaluated pairs. Since the data is synthetic and the split procedure is only partly documented, we treat these numbers as a promising demonstration and not as evidence of real-world performance, and we list the validation steps that should follow.**

***Index Terms*—Duplicate detection, entity resolution, record linkage, string similarity, blocking, Levenshtein distance, Jaro–Winkler, TF–IDF, machine learning, data quality.**

I. Introduction

Customer and entity data seldom comes from one clean source. It is typed into websites, collected by mobile apps and point-of-sale systems, logged at service desks, and carried over from older databases. When the same person or organisation is entered twice, the two copies rarely match exactly. A name may be misspelled, a phone number may pick up dashes, an address may be abbreviated or reordered, or a field may just be left empty. A plain equality check cannot tell that such records belong together, so the duplicates stay. They split customer histories, distort counts, add to storage and maintenance costs, and weaken any analysis done on top of the data.

Finding these duplicates is called record linkage or entity resolution: deciding whether two records describe the same real-world entity. It is hard because no single attribute can be trusted by itself. Names have typos, emails change, phone numbers get reformatted, and addresses come in different word orders. A dependable system has to look at several fields and put the evidence together, which is roughly what a careful human reviewer would do.

This paper presents a system built along those lines. It combines data preprocessing, candidate generation through blocking, multi-field similarity computation, supervised classification, duplicate clustering and a web-based analytics interface. We evaluated it on a generated customer dataset of 503 rows, and Table I gives the main numbers from that run.

TABLE I
KEY FIGURES OF THE EXPERIMENTAL RUN

Quantity	Value	Note
Input records	503	350 originals + 153 injected
Injected duplicates	153	65 exact, 88 near
Candidate pairs	605	After blocking
Flagged duplicate pairs	172	Predicted matches
Representative records	357	After clustering
Pairs in confusion matrix	182	Evaluated subset

Our main contributions are: (1) a modular pipeline that avoids unnecessary comparisons through blocking; (2) a multi-field similarity representation for structured customer records; (3) a side-by-side comparison of three supervised classifiers on one shared feature set; (4) a step that converts pairwise predictions into record clusters and a cleaned dataset; and (5) an interactive interface for inspecting matches and downloading results.

The rest of the paper is organised as follows. Section II reviews related work, and Section III states the research gap and the problem. Section IV describes the architecture. Sections V and VI cover the dataset, blocking and features, Section VII the classifiers and clustering, and Section VIII the implementation. Results are in Section IX, discussion and limitations in Section X, and Section XI concludes.

II. Related Work

Record linkage has a long history in statistics and databases. Fellegi and Sunter proposed a probabilistic framework in which the pattern of agreement and disagreement across attributes decides whether two records are linked [1]. The lesson that has lasted is that evidence from several fields should be weighed together, and no single attribute should decide on its own.

Approximate string comparison is the next building block. Levenshtein distance counts the insertions, deletions and substitutions needed to turn one string into another [2]. Jaro similarity was designed for record linkage and works well on short strings with transposed characters [3], and Winkler's prefix adjustment gives extra credit to names that start alike [4]. A comparison of string distance metrics for

name matching found that the best choice depends on the data [14]. Token-based measures give a different view. Jaccard similarity compares token sets and so tolerates reordering, while TF–IDF weights terms by how distinctive they are and compares the resulting vectors using cosine similarity [5], [15]. Christen gives a broad treatment of the whole linkage pipeline [6].

Scalability depends on how candidates are generated. Comparing every pair among *n* records requires

$$n(n - 1) / 2 \quad (1)$$

comparisons, and that number grows quadratically. Blocking and indexing shrink the search space by grouping records that share selected attributes [7], [8]. The price is that blocks that are too strict can throw away true duplicates before the classifier ever sees them. Learning-based approaches then treat each candidate pair as a feature vector. Bilenko and Mooney studied learnable string similarity for duplicate detection [9]. Logistic Regression is an interpretable baseline, Random Forest can capture non-linear interactions [10], and XGBoost gives efficient gradient-boosted trees for tabular data [11]. Deep and pretrained-language-model matchers go further still [12], [13].

We chose classical similarity functions and supervised classifiers over a large neural matcher on purpose. This keeps the system easy to inspect and realistic for a student project, and it leaves room for transformer-based matching, active learning and human review later. Table II shows how each strand of prior work is used here.

TABLE II
PRIOR WORK AND ITS ROLE IN THIS PROJECT

Topic	Ref.	Contribution used here
Probabilistic linkage	[1]	Combine evidence from many fields
Edit distance	[2]	name/email/address Levenshtein
Jaro and Jaro–Winkler	[3], [4]	Short-string, prefix-aware name match
String metric comparison	[14]	Justifies using several metrics
TF–IDF and set similarity	[5], [15]	Token overlap and weighted cosine
Linkage pipeline design	[6]	End-to-end workflow design
Indexing and blocking	[7], [8]	Blocking key + neighbour pairs
Learnable similarity	[9]	Supervised pair classification
Random Forest, XGBoost	[10], [11]	Non-linear classifiers compared
Deep matching	[12], [13]	Listed as future extension

III. Research Gap and Problem Formulation

Looking at the literature and at how deduplication is done in practice, a few gaps keep showing up. Exact-match rules fail on near duplicates caused by typos, punctuation, capitalisation or abbreviations. A single similarity function ignores evidence held in other fields. Comparing every pair becomes expensive as the data grows. And a bare yes/no prediction is not enough in real use, because people want to see both records, understand why they were matched, and review uncertain cases before anything is merged.

There is another gap between model evaluation and data cleaning. Some systems report classification metrics but never turn the predicted pairs into entity groups or produce a cleaned dataset. A useful workflow should go from raw input to reviewable pairs to a downloadable result. It should also keep the count of pairs separate from the count of rows removed, since one record can take part in several duplicate relationships. Table III links each gap to our design response.

TABLE III
RESEARCH GAPS AND DESIGN RESPONSES

Gap	Response in this work
Exact match misses near duplicates	Normalisation + approximate similarity
One metric sees one kind of error	Character, token, vector and exact features
All-pairs cost grows quadratically	Blocking + neighbouring-name safety net
Predictions are hard to trust	Searchable pair table with scores
Metrics without cleaning	Union-find clusters + cleaned CSV
Pairs vs. rows confused	Both counts reported separately

Formally, let the input be $R = {}_{r\{1}, r_2, \ldots, r_n\}$, where every record carries attributes such as name, email, phone and address. We want the set of pairs $M = {}_{(r\{i}, r_j)\}$ that refer to the same real entity. For every candidate pair we build a feature vector $\mathbf{x}_{ij}$ from field-level comparison functions, and a classifier $f(\mathbf{x}_{ij})$ then labels the pair as duplicate or not. In practice the pipeline needs four things: normalised data, a way to generate candidates, evidence combined across fields, and a final step that consolidates records.

The project objectives and the modules that deliver them are listed in Table IV. The scope is structured customer-record deduplication. Because the evaluation uses generated data, it should not be taken as proof of performance on every real customer database.

TABLE IV
PROJECT OBJECTIVES AND REALISING MODULES

No.	Objective	Module
O1	Normalise attributes, find incomplete rows	Preprocessing
O2	Generate candidates (key + neighbouring names)	Blocking
O3	Compute Lev., JW, Jaccard, TF–IDF, exact features	Similarity engine
O4	Train and compare LR, RF and XGBoost	Model layer
O5	Tune by grid search, 5-fold CV (as designed)	Model layer
O6	Produce pair report and cleaned dataset	Consolidation
O7	Web interface for upload, inspection, download	Dashboard

IV. System Architecture

The system is a chain of loosely coupled stages (Fig. 1). The input layer accepts a CSV file or generates a synthetic customer dataset. Preprocessing standardises text and phone values. Candidate generation groups records that are likely to be related and forms pairs for detailed comparison. The feature layer computes similarity values, the model layer predicts whether a pair is a duplicate, and the consolidation layer groups linked records and picks one representative. The presentation layer then shows the results and offers downloads.

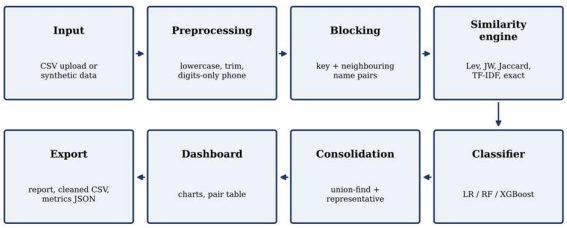


Fig. 1. Functional architecture of the similarity-based duplicate detection pipeline.

We deliberately keep pair-level matching separate from record-level cleaning. Pair-level matching asks whether two records probably describe the same entity. Consolidation then joins the predicted links into connected components. This separation matters because one entity can appear in three or more rows and produce several pairwise links, so the number of flagged pairs is not the number of rows removed. Table V lists the functional modules and Table VI the software stack.

TABLE V
FUNCTIONAL MODULES

Module	Responsibility
Dataset ingestion	Read records, validate required columns
Data preparation	Handle missing values, standardise strings and phones
Blocking	Build candidate groups from name and phone
Similarity engine	Character, token, vector, exact-match features
Model evaluation	Train and evaluate the classifiers
Duplicate clustering	Merge links, choose representatives
Dashboard and export	Charts, metrics, pair details, CSV output

TABLE VI
SOFTWARE STACK

Component	Role
Python	Core language for processing and ML
pandas, NumPy	Tabular data manipulation
scikit-learn	Preprocessing, TF–IDF, model utilities
XGBoost	Gradient-boosting classifier
Python-Levenshtein, jellyfish	Edit-distance and Jaro–Winkler
matplotlib, seaborn	Visual analysis
Flask	Web interface, async runs, progress
Streamlit, Plotly	Dashboard add-on (upload, threshold, charts)

The system is built to run locally for experiments. A production version would need authentication, audit logging, access restrictions, secure storage and a review workflow for uncertain matches, as discussed in Section X.

V. Dataset and Preprocessing

A. Dataset

To test duplicate detection under controlled conditions we used a generated customer dataset. The generator produces 503 rows: 350 original records and 153 injected duplicates, of which 65 are exact copies and 88 are near duplicates with one or more altered attributes. It picks 122 underlying source records to duplicate, and 18 rows have at least one missing field. The result is a mix of easy exact-match cases and harder altered ones (Table VII).

TABLE VII
GENERATED DATASET COMPOSITION

Property	Count
Total rows	503
Original records	350
Injected duplicates	153
– exact copies	65
– near duplicates	88
Source records duplicated	122
Rows with a missing field	18

Each record has an identifier, name, email, phone, address and a duplicate_of field. The duplicate_of field gives the ground truth for building labels and for evaluation. It is never used as an input feature, so there is no target leakage.

B. Preprocessing

Missing text values are turned into empty strings for comparison, and the fields are normalised as shown in Table VIII. For example, “999-673-4137” and “9996734137” become the same string after normalisation, so formatting alone can no longer hide a match.

TABLE VIII
NORMALISATION RULES BY FIELD

Field	Treatment	Effect
Name	Lowercase, collapse spaces, drop symbols	Ignores case/spacing
Email	Lowercase, collapse spaces, drop symbols	Ignores case
Address	Lowercase, collapse spaces, drop symbols	Ignores case/punctuation
Phone	Keep digits only	Ignores separators
Missing	Set to empty string; row flagged	Completeness check

Missing values need some care. Two empty fields must not count as an exact match, since having no information is not evidence that two records are the same person. The comparison should either record missingness separately or return a neutral score. A production version should also keep the original raw values next to the normalised ones, so that the cleaning stays auditable and reversible.

The pair table stores both record identifiers, the original attribute values, the feature scores, a combined score, the reference label and the predicted label. Our candidate_pair_scores.csv has 605 rows and 15 columns and is the main artifact for pair-level evaluation. Synthetic data is probably simpler than real data, which can contain multilingual names, shared household phones, reused emails and correlated missing values, so independent evaluation on real data is needed before deployment.

VI. Candidate Generation and Features

A. Blocking

Comparing all 503 records with each other would need $503 \times 502 / 2 = 126{,}253$ pairs by (1), and most of them are clearly unrelated. Our blocking strategy builds a key from the sorted first letters of the name tokens together with the first three digits of the normalised phone number, and records that fall in the same block become candidate pairs. To catch likely matches that straddle a strict block boundary, we also add neighbouring records after sorting by normalised name. This gave 605 candidate pairs, which Table IX compares with the all-pairs baseline.

TABLE IX
SEARCH-SPACE REDUCTION BY BLOCKING

Measure	Value
All possible pairs (n = 503)	126,253
Candidate pairs after blocking	605
Share of all pairs kept	≈ 0.48%
Pairs avoided	≈ 99.52%

The last two rows are simple arithmetic on the reported counts. They measure efficiency only: a smaller candidate set does not tell us how many true duplicates survived blocking. That needs blocking recall (pair completeness), which our current outputs do not report. The exact key construction sits in a feature-engineering module that is not part of the code described here, so we report the observed

605 pairs and make no claim about the key beyond the description above.

B. Similarity Features

For each candidate pair (A, B) the feature engine compares the matching attributes. Normalised Levenshtein similarity is

$$S_{lev}(a, b) = 1 - d(a, b) / \max(|a|, |b|) \quad (2)$$

where d(a, b) is the edit distance. Two empty strings are handled separately, and values close to 1 mean high similarity. Jaro–Winkler adds a prefix bonus to Jaro similarity and suits personal names, whose first characters are often stable, although a shared prefix can mislead if the measure is used alone. Jaccard similarity,

$$J(A, B) = |A \cap B| / |A \cup B| \quad (3)$$

compares token sets and so tolerates reordering; “Road 12, Central Area” and “Central Area, Road 12” contain the same tokens. TF–IDF maps text to weighted vectors, giving less weight to common words, and cosine similarity compares them:

$$\cos(u, v) = (u \cdot v) / (\|u\| \, \|v\|) \quad (4)$$

We apply TF–IDF cosine over the combined textual fields. Exact-match indicators complement the approximate scores: phone_exact_match and email_exact_match equal one when the normalised values are identical. They are strong evidence but never proof, since contact details can be shared, recycled or missing. Table X lists the feature table we implemented.

TABLE X
PAIRWISE FEATURE CATALOGUE

Feature	Measure	Captures
name_levenshtein	Edit distance	Small name typos
name_jaro_winkler	Jaro–Winkler	Stable name prefix
email_levenshtein	Edit distance	Email variants
address_levenshtein	Edit distance	Address typos
address_jaro_winkler	Jaro–Winkler	Address prefix
name_jaccard	Token overlap	Reordered name parts
address_jaccard	Token overlap	Reordered address parts
phone_exact_match	Binary	Same normalised phone
email_exact_match	Binary	Same normalised email
tfidf_cosine	Vector cosine	Weighted text overlap
combined_score	Weighted sum	Overall pair evidence

A weighted combined score can be written in general form as

$$C(A, B) = \Sigma_k \, w_k \, s_k(A, B) \quad (5)$$

where the w_k are non-negative weights and the s_k are normalised comparison signals. The actual weights, scaling and threshold are specific to the implementation and are not visible in the dashboard, so they should be taken from the source code when reproducing the experiment. We do not assign numerical weights here.

VII. Classification and Consolidation

A. Classifiers

Every candidate pair becomes a numerical vector $\mathbf{x}_{ij}$ with a label saying whether the two records are the same entity. Logistic Regression, Random Forest and XGBoost are all evaluated on this same representation, so any differences come from the learners and not from the evidence they are given. Logistic Regression estimates

$$P(y = 1 \mid \mathbf{x}) = 1 / (1 + \exp(-(\beta_0 + \boldsymbol{\beta}^T \mathbf{x}))) \quad (6)$$

and a decision threshold turns the probability into a label. Random Forest averages many decision trees trained on bootstrapped samples and random feature subsets, and it can learn, for example, that a high name similarity counts for more when the phone number also matches exactly. XGBoost builds its trees one after another, each correcting the errors of the ensemble so far, and uses regularisation to keep complexity under control. Table XI compares the three.

TABLE XI
CLASSIFIER COMPARISON

Model	Strength	Consideration
Logistic Regression	Interpretable baseline	Linear boundary
Random Forest	Non-linear interactions	Less transparent
XGBoost	Strong on tabular data	Sensitive to tuning

The original design uses a 70/30 train–test split and grid search with five-fold cross-validation. Tuning should use only the training partition so that the test set stays untouched for the final evaluation. Pair-level splitting needs care as well. If records of the same underlying entity end up in both partitions, leakage makes performance look better than it would on unseen entities, so a stronger evaluation would split by entity or source record.

B. Record-Level Consolidation

Predicted duplicate pairs are turned into groups with a union-find structure. Each record starts in its own set, and every predicted pair merges the sets of its two records. The resulting connected components are our candidate entity clusters. If A is linked to B and B to C, all three end up in one cluster even if the A–C pair was never predicted. For each cluster the implementation keeps the record with the fewest missing fields as the representative and leaves the others out of the cleaned dataset.

This is simple and reproducible, but one false-positive edge can merge two distinct entities. Picking the most complete row also ignores recency, source authority and conflicting values. A production version should therefore add manual review, confidence thresholds, field-level survivorship rules and a reversible merge log. The steps we follow, as seen in the dashboard source, are summarised as Algorithm 1 in Table XII.

TABLE XII
ALGORITHM 1: CLASSIFICATION AND CONSOLIDATION

Step	Action
1	Validate record_id, name, email, phone, address; reject duplicate or non-numeric IDs
2	Preprocess records; keep source attributes for display
3	Generate candidate pairs (blocking + adjacent names)
4	Compute the pairwise feature vector for each candidate
5	With enough labels: split, train, evaluate, pick best model, predict. Otherwise: apply threshold τ to the combined score
6	Build duplicate report from predicted matches and join original attributes
7	Create one disjoint set per record; union the endpoints of each predicted pair
8	Count missing fields; select one representative per component
9	Show metrics, charts, searchable pairs and CSV downloads

Most of the cost sits in the early stages. Exhaustive pairing is $O(n^2)$ in the number of pairs, while blocking aims for m far below $n(n-1)/2$. With bounded field lengths, feature construction is roughly $O(m \cdot L)$, where L is the average cost of one string comparison, and union-find does

m unions over n records in near-linear amortised time. We did not run a formal runtime benchmark.

VIII. Implementation and Dashboard

The system is a modular Python application. A data layer loads and validates the CSV and creates the normalised fields, a candidate layer applies blocking, a similarity layer computes the string and vector features, a model layer trains the classifiers and predicts pair labels, and a consolidation layer forms clusters and exports the representatives. The Flask interface can generate a synthetic dataset or accept a CSV of names, emails, phones and addresses, and it runs asynchronously with progress updates so the browser stays responsive. A Streamlit and Plotly add-on offers dataset selection or upload, a similarity-threshold control for unlabelled data, pair inspection, model comparison, confusion-matrix display and downloads. The imported source modules were not part of the add-on archive, so the blocking keys, feature weights, split settings and hyperparameters could not be independently verified.

The dashboard, titled “Duplicate Detection Intelligence Studio”, is meant to make the model's decisions visible instead of hiding them. Table XIII lists its components and Table XIV the output artifacts.

TABLE XIII
DASHBOARD COMPONENTS

Component	Purpose
Summary indicators	Input records, candidate pairs, flagged pairs, unique records
Score histogram	Flagged vs. non-flagged similarity
Reduction chart	Input vs. unique records
Model table + chart	Compare three classifiers
Confusion matrix	Error counts of the best model
Searchable pair table	Inspect names, emails, phones, addresses, scores
Threshold slider	Unlabelled data only

TABLE XIV
OUTPUT ARTIFACTS

File	Content
candidate_pair_scores.csv	All candidate pairs with features, labels
duplicate_records_report.csv	Predicted duplicate pairs, readable values
cleaned_unique_dataset.csv	One representative per cluster
model_performance_metrics.json	Classifier metrics
Charts	Model comparison, confusion matrix, score distribution, dataset size

Keeping the pair scores, the duplicate report and the cleaned dataset as separate outputs helps traceability, because a reviewer can tell model evaluation apart from the final consolidation result. For uploads without a usable duplicate_of field, the application applies the selected combined-score threshold and reports no supervised accuracy, precision or recall. Input validation checks the required columns, rejects repeated identifiers, converts identifiers to integers, and casts text fields to strings after filling missing values.

IX. Experimental Results

Figs. 2–6 show the dashboard from the documented run. The 503 input records produced 605 candidate pairs, 172 of which were flagged, leaving 357 unique records after consolidation.

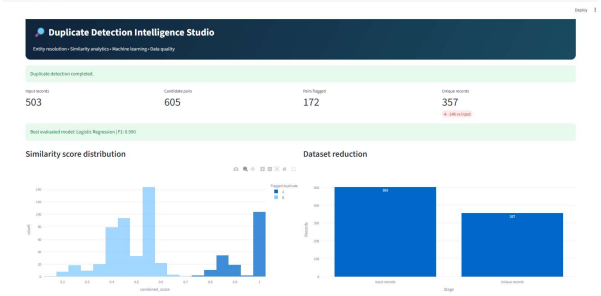


Fig. 2. Duplicate Detection Intelligence Studio dashboard overview.



Fig. 3. Model evaluation comparison for three classifiers.

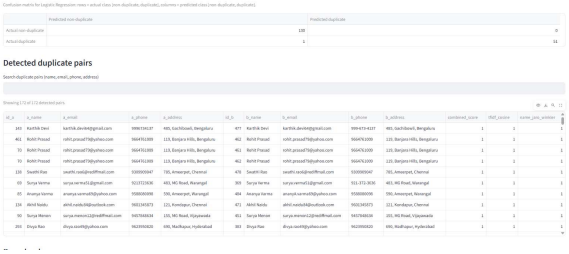


Fig. 4. Confusion matrix and detected duplicate-pair table.

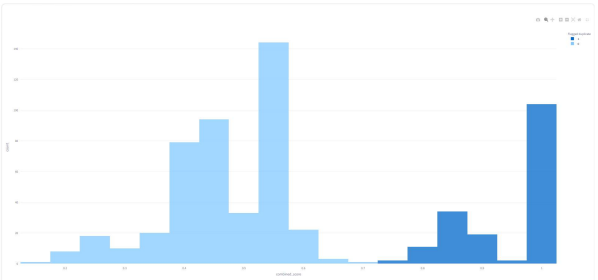


Fig. 5. Distribution of combined similarity scores.

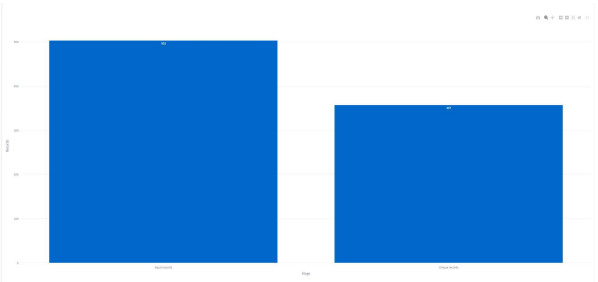


Fig. 6. Dataset reduction from 503 input records to 357 unique records.

A. Record Reduction

Record reduction and flagged-pair rate measure different things, as Table XV shows. The first is a reduction in record count, and the second is the share of candidate pairs classified as duplicates. Note that 172 links reduced the row count by only 146, because links overlap inside clusters: a cluster of three records can produce several links while removing just two rows. The figure of 146 should not be read as the true number of duplicate entities unless the ground-truth clusters are checked independently.

TABLE XV
REDUCTION AND FLAGGING STATISTICS

Measure	Calculation	Result
Records removed	503 − 357	146
Record reduction	146 / 503 × 100	29.02%
Flagged-pair rate	172 / 605 × 100	28.43%

B. Classifier Performance

The dashboard reports identical rounded metrics for all three models (Table XVI) and names Logistic Regression as the best one. Because the values are rounded to three decimals, the screenshot cannot show whether the unrounded scores are identical or how the tie was broken. We therefore say the models performed similarly instead of calling one of them superior.

TABLE XVI
DISPLAYED PERFORMANCE OF THE CLASSIFIERS

Model	Acc.	Prec.	Rec.	F1
Logistic Regression	0.995	1.000	0.981	0.990
Random Forest	0.995	1.000	0.981	0.990
XGBoost	0.995	1.000	0.981	0.990

The confusion matrix for Logistic Regression (Table XVII) covers 182 candidate pairs. Accuracy is (130 + 51)/182 = 0.9945, precision is 51/(51 + 0) = 1.0000, recall is 51/(51 + 1) = 0.9808, and the harmonic mean of precision and recall is about 0.9903. All of these round to the dashboard values. How the 182 pairs were selected cannot be established from the available evidence.

TABLE XVII
CONFUSION MATRIX (LOGISTIC REGRESSION)

	Pred. non-dup.	Pred. dup.
Actual non-duplicate	130 (TN)	0 (FP)
Actual duplicate	1 (FN)	51 (TP)

No false positive appears in the displayed sample, and one true duplicate was missed. The two kinds of error do not cost the same (Table XVIII), so the threshold should be chosen to suit the intended application. The score histogram shows non-duplicates at low and middle combined scores and flagged pairs at high scores, with a clear cluster near 1.0. This separation fits a composite similarity feature, but it is not independent proof that every flagged pair is correct.

TABLE XVIII
COST OF EACH ERROR TYPE

Error	Consequence	Mitigation
False positive	Distinct people merged	Higher threshold, review
False negative	Duplicate row survives	Lower threshold, rerun

The result is promising for the generated sample, but it does not show that the system generalises to real customer datasets. That would need independent labels, leakage-safe splitting, repeated validation and external data.

X. Discussion, Limitations and Validity

The results show the value of combining several comparison signals. Character-level similarity catches small spelling differences, token similarity tolerates reordering, vector similarity captures overlap across combined text, and exact phone and email matches add strong evidence. A classifier can learn how these signals interact instead of applying one rule to everything. We see the system as an experimental decision-support tool. It demonstrates an end-to-end architecture and gives high scores on a synthetic dataset, but it needs more validation before operational use. Table XIX lists the main threats to validity.

TABLE XIX
THREATS TO VALIDITY

Threat	Why it matters	Remedy
Synthetic-data bias	Corruptions may be simpler than real ones	Real labelled data
Entity leakage	Same entity in train and test inflates scores	Entity-level split
Candidate selection	Classifier only sees pairs that survive blocking	Measure blocking recall
Rounded metrics	Ties cannot be resolved	Report unrounded scores
Unknown split protocol	Seed and split not shown	Log full configuration
Missing source modules	Weights, keys unverified	Publish full source
Cluster propagation	One bad link merges entities	Review, reversible merges

Blocking needs its own evaluation. Pair completeness (blocking recall) is the share of all known true duplicate pairs that appear in the candidate set, and pair quality is the share of candidate pairs that are true matches. A strict rule gives a small, clean candidate set but may miss matches, while a permissive rule keeps more matches but puts more load on the classifier. A key built from name initials and phone prefixes may also fail when both fields are corrupted. Future work should report both measures.

Several limitations follow from the evidence we have. The dataset is generated, the exact combined-score weights and threshold are not visible, the unrounded per-model metrics and the random seed are not shown, and a pair-level random split may leak entities. Synthetic duplicates are made by copying a row and applying a limited set of transformations, so a model may simply learn those transformations. The reported figures should therefore be read as results on the current generated evaluation sample and not as a general expected accuracy.

For real-world use, Table XX summarises what a production system would still need. Customer records are personal data, so access should be restricted, data protected in transit and at rest, full contact details kept out of logs, and a retention policy defined. Duplicate detection should never delete source records without review, especially where billing, service or legal records are involved. The dashboard already warns that a flagged pair is only a candidate and recommends reviewing borderline matches, and that safeguard is worth keeping.

TABLE XX
PRODUCTION READINESS CHECKLIST

Area	Current state	Recommended addition
Security	Local experiment	Authentication, roles, TLS
Auditability	CSV exports	Log model, threshold, reviewer
Review	Searchable table	Approve / reject / defer queue
Merges	Keep fewest-missing row	Field-level survivorship, undo
Scale	503 records	Batch, DB-backed, incremental
Monitoring	Score histogram	Drift and flag-rate tracking
Reproducibility	Partial	JSON manifest, seeds, versions

Error analysis should also be done by attribute. A false negative may occur when a name changes a lot and the contact fields are incomplete, while a false positive may occur when two different people share a common name and

address, or when an email or phone number is reused. Grouping errors by corruption type, such as name typo, phone formatting, missing email or address variation, would show where the features need work. Exporting false-positive and false-negative examples together with their ground-truth labels would make this analysis routine.

XI. Conclusion and Future Work

This paper presented a similarity-based duplicate record detection system for structured customer data. It combines normalisation, blocking, multi-field similarity features, supervised classification, duplicate clustering and an interactive dashboard, using Levenshtein, Jaro–Winkler, Jaccard, TF–IDF cosine and exact phone and email signals. In the documented run, 503 input records produced 605 candidate pairs, 172 flagged pairs and 357 representative records, a reduction of 146 rows or about 29.0%. All three classifiers showed 0.995 accuracy, 1.000 precision, 0.981 recall and 0.990 F1, with 130 true negatives, no false positives, one false negative and 51 true positives. These are strong results on the displayed sample, but they should be read in light of the synthetic data and the limited validation evidence.

The most important next step is a stronger experiment and not another model: leakage-safe splitting, external data, blocking-recall measurement and a reviewable merge process. Table XXI lays out the planned directions.

TABLE XXI
FUTURE WORK ROADMAP

Direction	Planned work
Evaluation	Real datasets, human labels, entity-level splits, repeated stratified validation, PR curves, confidence intervals
Blocking	Measure pair completeness, improve candidates
Modelling	Learn feature weights, calibrate thresholds, try transformer matchers
Human review	Active learning, approve / reject workflow
Consolidation	Survivorship rules, reversible merges, audit logs
Scalability	Incremental matching against existing entities

Overall, the system gives a practical foundation for structured-record deduplication and shows how approximate string matching and machine learning can be combined in a usable data-quality workflow.

Acknowledgment

The authors thank Ms. Bonagani Prathusha, Assistant Professor, Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, for her guidance and academic support. We also thank the department and the institution for providing the academic environment for this project.

References

[1] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *J. Amer. Statist. Assoc.*, vol. 64, no. 328, pp. 1183–1210, 1969.

[2] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Phys. Doklady*, vol. 10, pp. 707–710, 1966.

[3] M. A. Jaro, "Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida," *J. Amer. Statist. Assoc.*, vol. 84, no. 406, pp. 414–420, 1989.

[4] W. E. Winkler, "String comparator metrics and enhanced decision rules in the Fellegi-Sunter model of record linkage," in *Proc. Sect. Survey Res. Methods, Amer. Statist. Assoc.*, 1990, pp. 354–359.

[5] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. New York, NY, USA: McGraw-Hill, 1983.

[6] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Berlin, Germany: Springer, 2012.

[7] P. Christen, "A survey of indexing techniques for scalable record linkage and deduplication," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 9, pp. 1537–1555, 2012.

[8] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Comput. Surv.*, vol. 53, no. 2, 2020.

[9] M. Bilenko and R. J. Mooney, "Adaptive duplicate detection using learnable string similarity measures," in *Proc. ACM SIGKDD*, 2003, pp. 39–48.

[10] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[12] S. Mudgal *et al.*, "Deep learning for entity matching: A design space exploration," in *Proc. ACM SIGMOD*, 2018, pp. 19–34.

[13] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *arXiv:2004.00584*, 2020.

[14] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "A comparison of string distance metrics for name-matching tasks," in *Proc. IJCAI*, 2003, pp. 73–78.

[15] A. Arasu, V. Ganti, and R. Kaushik, "Efficient exact set-similarity joins," in *Proc. VLDB*, 2006.